Version 1.1

January 2026

DATA MINING

MODULE 1 - DATA STRUCTURE IN PYTHON

Authored by:

Ir. Yufis Azhar, S.Kom., M.Kom.

Haidar Dimas Heryanto

Muhammad Aunul Hakim

INFORMATICS LABORATORY

University of Muhammadiyah Malang

TABLE OF CONTENTS

TABLE OF CONTENTS **2**
INTRODUCTION **3**
PRACTICAL OBJECTIVES 3
PREPARATION 3
KEYWORDS 3
1. List and Tuple **4**
1.1 List 4
1.2 Tuple 9
2. NumPy and Array **11**
2.1 Array NumPy vs Python List 11
2.2 Creating Arrays in NumPy 11
2.3 Array Properties 12
2.4 Mathematical Operations on Arrays 13
2.5 Indexing and Slicing 15
2.6 Reshaping 15
2.7 Concatenating Arrays 16
3. Pandas Dataframe **18**
3.1 Creating a Dataframe from a Simple Array 19
CODELAB (20%) **21**
CODELAB 1 21
CODELAB 2 21
CODELAB 3 21
LAB ASSIGNMENT (80%) **22**
ASSIGNMENT 1 22
ASSIGNMENT 2 22
ASSIGNMENT 3 (EVEN STUDENT ID/NIM) 22
ASSIGNMENT 3 (ODD STUDENT ID/NIM) 23
GRADING CRITERIA & DETAILS **24**

INTRODUCTION

PRACTICAL OBJECTIVES

1. Students are able to understand the basics of Python programming language.
2. Students are able to understand data structure types in Python programming language.
3. Students are able to understand lists, arrays, tuples, and dataframes.

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson

رَضِيْتُ بِاللهِ رَبًّا وَبِالإِسْلَامِ دِيْنًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُوْلًا. رَبِّ زِدْنِيْ عِلْمًا وَارْزُقْنِيْ فَهْمًا

2. Hardware:
   - Laptop/Computer
3. Software (Choose one):
   - Visual Studio Code, Google Collaboratory, PyCharm, Jupyter Notebook
4. Tutorial
   - Using Google Colab: https://youtu.be/tpDjhSzJor4?si=bFjSP_mf3oB5x520
   - Using Jupyter in Visual Studio Code: https://youtu.be/HOg13z1wI1A?si=RhEY2QY5bir-dzEH
   - Using Jupyter Notebook: https://youtu.be/HQaCF2F_MRo?si=k6ubuGuCg-VS0tQe
   - Install Anaconda: https://youtu.be/XyudQ3B3OWI?si=7qugN_vTxAM30qDt
5. Source Code (Google Colaboratory):
   - *https://bit.ly/module1-colab*
6. Strong motivation and commitment to learning 🤗

KEYWORDS

Python Programming, Data Structures, List, Array, Tuple, Dataframe

1. List and Tuple

In the previous semester, you have learned about Functional Programming which discusses data types in Python, including sequence types. A sequence is a collection of values accessed through an index and has several main data types, such as arrays. However, in Python, sequence data types are slightly different from arrays in general. Sequence data is more dynamic than arrays; we can add as many elements as we want without size limitations like in arrays. In addition, sequences can also store elements with different data types.

In this module, we will revisit two very frequently used sequence data types: List and Tuple. Both have similar features but with certain characteristics that differentiate them.

1.1 List

Lists in Python are one of the most flexible and frequently used sequence data types. A list contains an ordered (indexed) collection of elements and can be changed (mutable). Here are some important points about lists in Python:

a. Mutable (Can Be Modified)

We can still make changes to data in a list because of its mutable nature, meaning its elements can be modified after the list is created.

b. Ordered (Sequential)

Ordered here means that to access data in a list, it will start sequentially according to the order of its index.

Example:

```
list = ['uno', 'dos', 'tres', 'cuatro', 'cinco']
print(list[0])
print(list[1])
print(list[2])
```

Output:

```
••• uno
    dos
    tres
```

c. Heterogeneous (Many Data Types)

Lists are not rigid, meaning they can store values of various data types, as shown in the example below:

Example:

```
list = ["data", 7, 1.25, 8, "mining", 5, False]
print(list)
```

Output:

```
... ['data', 7, 1.25, 8, 'mining', 5, False]
```

d. Dynamic

Due to its mutable nature, lists allow operations on data. Operations that can be used on lists:

- **Adding Items**

1. **extend()**

   Adds all iterable elements (such as a list, tuple, or string) one by one to the end of the list.

   Example:

```
list = [1, "Hiroshi", True]
list.extend(["Kazuki", 7, 2.15])
print(list)
```

   Output

```
... [1, 'Hiroshi', True, 'Kazuki', 7, 2.15]
```

2. **append()**

   Adds a single element treated as one single object to the end of the list.

   Example:

```
list = [1, "Osamu", True]
list.append(["Ren", 7, 2.15])
print(list)
```

   Output:

```
... [1, 'Osamu', True, ['Ren', 7, 2.15]]
```

3. **insert()**

Adds a new list to a specified position. In the code below, we will insert a new list before index 2, which is before the True element.

Example:

```
list = [1, "Osamu", True]
list.insert(2, ["Mugen", 7, 2.15])
print(list)
```

Output:

```
... [1, 'Osamu', ['Mugen', 7, 2.15], True]
```

- **Removing Items**

1. **remove()**

Removes the first item in the list that matches the given value.

Example:

```
fruits = ["Dewandaru", "Feijoa", "Abiu", "Black
Sapote"]
fruits.remove("Black Sapote")
print(fruits)
```

Output

```
... ['Dewandaru', 'Feijoa', 'Abiu']
```

If the remove command is executed again, the current value item will be removed.

Example:

```
fruits.remove("Abiu")
print(fruits)
```

Output:

```
... ['Dewandaru', 'Feijoa']
```

2. **pop(index)**

Removes an item by its index and returns the removed item. If no index value is given, the last item will be removed.

Example:

```
numbers = [10, 20, 30, 40, 50, 60, 70]
```

```
without_index = numbers.pop()
print("Pop without index:", numbers)
print("Removed:", without_index)

index = numbers.pop(2)
print("Pop with index:", numbers)
print("Removed:", index)
```

Output:

```
Pop without index: [10, 20, 30, 40, 50, 60]
Removed: 70
Pop with index: [10, 20, 40, 50, 60]
Removed: 30
```

3. **clear()**
   Removes all items from the list, making the list empty.

   Example:

```
names = ["Kazuki", "Takumi", "Jinpei"]
names.clear()
print(names)
```

Output:

```
[]
```

- **Finding Items**
  1. **index(item)**
     Searches for the index of the first item that matches the given value.

     Example:

```
colors = ["Aka", "Ao", "Midori", "Ao"]
index = colors.index("Ao")
print(index)
```

Output:

```
1
```

- **Counting Items**
  1. **count(item)**
     Counts the number of occurrences of an item in the list.

     Example:

```
scores = [10, 20, 10, 30, 10, 20, 10, 70, 10]
count = scores.count(10)
print(count)

# Output: 5
```

     Output:

```
5
```

- **Sorting Items**
  1. **sort()**
     Sorts items in the list in ascending order by default, or descending if the parameter reverse=True is used.

     Example:

```
numbers = [10, 20, 10, 30, 10, 20, 10, 70, 10]
numbers.sort()  # Ascending
print(numbers)

numbers.sort(reverse=True)  # Descending
print(numbers)
```

     Output:

```
[10, 10, 10, 10, 10, 20, 20, 30, 70]
[70, 30, 20, 20, 10, 10, 10, 10, 10]
```

- **Reversing Items**
  1. **reverse()**
     Reverses the order of elements in a list without regard to element values.

Example:

```
letters = ["a", "b", "c", "d", 1, 4, True, 2, 3, 1]
letters.reverse()
print(letters)
```

Output:

```
[1, 3, 2, True, 4, 1, 'd', 'c', 'b', 'a']
```

1.2 Tuple

A tuple is an ordered sequence of objects in Python that is immutable. Tuples are similar to lists, but the main difference is that elements within a tuple cannot be changed after they are defined. Tuples use parentheses '()', while lists use square brackets '[]'. Creating a tuple is very easy; we just need to write values separated by commas. Here are some important points about tuples in Python:

a. Immutable

Once successfully created, a tuple cannot be changed, including adding or deleting elements within it.

Example:

```
tuple = ("a", "b", "c", "d", "e")
print(tuple)

tuple = ("z",) + tuple[1:]
print(tuple)
```

Output:

```
('a', 'b', 'c', 'd', 'e')
('z', 'b', 'c', 'd', 'e')
```

b. Heterogeneous (Many Data Types)

Tuples allow for different data types, similar to Lists.

Example:

```
tuple = ("data", 7, 1.25, 8, "mining", 5, False)
print(tuple)
```

Output:

```
... ('data', 7, 1.25, 8, 'mining', 5, False)
```

c. Ordered

Tuples are ordered like Lists, meaning we can access individual data elements using their respective indices.

Example:

```
tuple = (1, 2, 3, 4, 5)
print(tuple[1])
print(tuple[2])
print(tuple[3])
```

Output:

```
... 2
    3
    4
```

d. Indexable

Each item in a tuple can be accessed using its index.

Example:

```
t = (10, 20, 30, 40)
print(t[0])
print(t[3])
```

Output:

```
... 10
    40
```

e. Allow Duplicates

Tuples also allow data with the same value, similar to Lists, as they are influenced by indices.

Example:

```
tuple = (True, 7, "Taro", "Taro", 7, True)
print(tuple)
```

Output:

```
... (True, 7, 'Taro', 'Taro', 7, True)
```

2. NumPy and Array

NumPy (Numerical Python) is a Python library used to manage large arrays, including multi-dimensional arrays. This library also provides various mathematical functions to simplify complex calculations on arrays. Arrays in NumPy are grid-shaped data structures specifically designed to store numerical data. Compared to Python lists, NumPy arrays are faster and more efficient in performing calculations because they have been optimized.

2.1 Array NumPy vs Python List

Array NumPy	Python List
Fixed Size	Dynamic Size
Elements must have the same data type	Element types can be diverse
Faster element-wise mathematical operations	Does not directly support operations

2.2 Creating Arrays in NumPy

a. 1D Array

Creates a one-dimensional array used to store a list of linear data.

Example:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print(arr)
```

Output:

```
... [1 2 3 4 5]
```

b. 2D Array

Creates a two-dimensional array that resembles a matrix or data table with rows and columns.

Example:

```
matrix = np.array([[1, 2], [3, 4], [5, 6]])
print(matrix)
```

Output:

```
[[1 2]
 [3 4]
 [5 6]]
```

c. **3D Array**

Creates a three-dimensional array that can store complex data such as tensors in machine learning.

Example:

```
array_3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
print(array_3d)
```

Output:

```
[[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]
```

2.3 Array Properties

a. **Array Dimension (ndim)**

Provides information about the number of array dimensions, e.g., 1D, 2D, or 3D.

b. **Array Shape**

Indicates the number of elements in each dimension (e.g., a 2x3 array has a shape of (2, 3)).

c. **Number of Elements (Size)**

Returns the total number of elements in the array.

d. **Data Type (dtype)**

Describes the data type used by the array, such as int32 or float64.

Example:

```
array = np.array([[1, 2, 3], [4, 5, 6]])

print("Dimensions:", array.ndim)
print("Shape:", array.shape)
print("Number of Elements:", array.size)
```

```
print("Data Type:", array.dtype)
```

Output:

```
Dimensions: 2
Shape: (2, 3)
Number of Elements: 6
Data Type: int64
```

2.4 Mathematical Operations on Arrays

a. Element-wise Operations

Performs mathematical operations such as addition, multiplication, or exponentiation on array elements directly, element by element.

Example:

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a + b)  # Output: [5 7 9]
print(a * b)  # Output: [4 10 18]
print(a ** 2) # Output: [1 4 9]
```

Output:

```
[5 7 9]
[ 4 10 18]
[1 4 9]
```

b. Matrix Operations

Performs linear algebra operations useful for large-scale numerical data manipulation.

- **Matrix Addition and Substraction**

  Addition and substraction of two matrices can only be performed if both have the same size.

  Example:

```
import numpy as np

mat1 = np.array([[1, 2], [3, 4]])
mat2 = np.array([[5, 6], [7, 8]])
```

```
# Addition
print(mat1 + mat2)

# Substraction
print(mat2 - mat1)
```

Output:

```
...  [[ 6  8]
      [10 12]]
     [[4 4]
      [4 4]]
```

- **Matrix Multiplication**

  Matrix multiplication can be performed as:

  - **Element-wise Multiplication**

    Multiplies corresponding elements in two matrices (operator * in NumPy).

  - **Matrix Multiplication**

    Standard linear multiplication where elements are calculated as the result of multiplying rows and columns (using the np.dot() function or the @ operator).

    Example:

```
import numpy as np

mat1 = np.array([[1, 2], [3, 4]])
mat2 = np.array([[5, 6], [7, 8]])

# Element-wise Multiplication
print(mat1 * mat2)

# Matrix Multiplication
print(np.dot(mat1, mat2))
```

    Output:

```
...  [[ 5 12]
      [21 32]]
     [[19 22]
      [43 50]]
```

2.5 Indexing and Slicing

Like Python lists, NumPy arrays support accessing individual elements and subsets of elements using indices. This feature is very flexible, even allowing manipulation of partial data without affecting the original data.

Example:

```
array = np.array([1, 2, 3, 4, 5])

# Accessing elements
print(array[0])
print(array[-1])

# Slicing
print(array[1:4])
print(array[:3])
```

Output:

```
... 1
    5
    [2 3 4]
    [1 2 3]
```

2.6 Reshaping

Reshaping is used to change the shape of an array without changing its data. For example, you can change a 1D array to 2D or vice versa.

Main Functions:

- **reshape()**: Changes the shape of the array to the desired dimension.
- **flatten()**: Converts a multi-dimensional array into a 1D array.

Example:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])

# Convert to 2D array (2 rows, 3 columns)
reshaped_arr = arr.reshape(2, 3)
print("Reshaped Array:")
print(reshaped_arr)
```

```
# Convert back to 1D array
flattened_arr = reshaped_arr.flatten()
print("\nFlattened Array:")
print(flattened_arr)
```

Output:

```
•••  1
     5
     [2 3 4]
     [1 2 3]
```

2.7 Concatenating Arrays

Concatenating arrays means combining several arrays into one, either horizontally (row-wise) or vertically (column-wise).

Main Functions:

- **np.concatenate():** Concatenates arrays along a specified axis.
- **np.vstack()**: Stacks arrays vertically (rows increase).
- **np.hstack()**: Stacks arrays horizontally (columns increase).

Example:

```
arr1 = np.array([[1, 2], [3, 4]])
arr2 = np.array([[5, 6], [7, 8]])

# Merge vertically
vertical_concat = np.vstack((arr1, arr2))
print("Vertical Concatenation:")
print(vertical_concat)

# Merge horizontally
horizontal_concat = np.hstack((arr1, arr2))
print("\nHorizontal Concatenation:")
print(horizontal_concat)
```

Output:

```
Vertical Concatenation:
[[1 2]
 [3 4]
 [5 6]
 [7 8]]

Horizontal Concatenation:
[[1 2 5 6]
 [3 4 7 8]]
```

2.8 Splitting

Array splitting means dividing one array into several smaller arrays with a certain number of elements.

Main Functions:

- **np.split()**: Splits an array at specified indices.
- **np.vsplit()**: Splits vertically.
- **np.hsplit()**: Splits horizontally.

Example:

```
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])

# Divide horizontally into two parts
horizontal_split = np.hsplit(arr, 2)
print("Horizontal Splitting:")
print(horizontal_split[0])  # First part
print(horizontal_split[1])  # Second part

# Divide vertically into two parts
vertical_split = np.vsplit(arr, 2)
print("\nVertical Splitting:")
print(vertical_split[0])  # First part
print(vertical_split[1])  # Second part
```

Output:

```
Horizontal Splitting:
[[1 2]
 [5 6]]
[[3 4]
 [7 8]]

Vertical Splitting:
[[1 2 3 4]]
[[5 6 7 8]]
```

3. Pandas Dataframe

Pandas is a very useful Python library for data analysis and processing, suitable for handling datasets from medium to large sizes. In Pandas, there are two main types of data structures:

a. Series: A one-dimensional array that can store various types of data and has an axis label.

b. DataFrame: A two-dimensional data structure, similar to a table with rows and columns, that can store various types of data and also has axis labels.

Some key advantages and features of Pandas are:

a. Fast Data Loading: Can easily load data from various sources such as CSV files, Excel, or databases.

b. Efficient Data Manipulation: Offers various functions and methods for cleaning, sorting, and restructuring data in a highly effective way.

c. Data Merging: Allows easy merging and grouping of data, similar to SQL join operations.

d. Various Built-in Functions: Provides many functions for comprehensive data aggregation, transformation, and analysis.

3.1 Creating a Dataframe from a Simple Array

Creating a DataFrame from a Simple Array is one of the basic ways to use the Pandas library for data manipulation and analysis.

Example:

```
import pandas as pd

# Calling the dataframe constructor
dataframe = pd.DataFrame()
print(dataframe) # Empty dataframe

# Create a list containing strings
words = ['this', 'is', 'dataframe']

# Calling the dataframe constructor on a list
dataframe = pd.DataFrame(words)
print("\n", dataframe)
```

Output:

```
•••  Empty DataFrame
     Columns: []
     Index: []

                 0
     0       this
     1         is
     2  dataframe
```

3.2 Creating a Dataframe from a Dictionary

A Python dictionary is a data structure consisting of key-value pairs, where keys can be used as column names within a DataFrame. By using a dictionary, users can easily organize data and create a neat DataFrame, allowing for more efficient data manipulation and analysis.

Example:

```
import pandas as pd

myvar = pd.DataFrame()
print(myvar) # The dataframe is empty

mydataset = {
  'Name': ["Yui", "Mio", "Ritsu", "Tsumugi", "Azusa"],
  'Height': [156, 160, 154, 157, 150]
}

# Calling the dataframe constructor on a list
myvar = pd.DataFrame(mydataset)
print(myvar)
```

Output:

```
•••  Empty DataFrame
     Columns: []
     Index: []
           Name  Height
     0      Yui     156
     1      Mio     160
     2    Ritsu     154
     3  Tsumugi     157
     4    Azusa     150
```

CODELAB (20%)

CODELAB 1

a. Create 12 data tuples, each containing 4 int, float, and string data in random order.
b. Convert the tuple data into a list.
c. Add 2 new data points at the index corresponding to your last student ID (NIM) digit (if the last student ID/NIM digit is 0, then index 10).
d. Remove float data types from the list.
e. Sort the elements in the list in descending order.

CODELAB 2

a. Create a 2D array based on the last 6 digits of your student ID (NIM).
b. Perform addition, subtraction, and multiplication operations on the matrix you created.
c. Convert the matrix into a 1D shape.
d. Perform slicing on the matrix.

CODELAB 3

a. Create a series containing a dictionary that includes product names and product IDs.
b. Create a dataframe containing a dictionary with 4 columns, and each column contains 5 data points. There should be no similarities between students.
c. Add 1 new column.
d. Sort the dataframe, for example, alphabetically if the column is 'Name'.

LAB ASSIGNMENT (80%)

ASSIGNMENT 1

Create a function that accepts a list as a parameter and performs the following operations:

a. Add 10 random numeric elements (int and float).
b. Remove elements at the index according to your student ID (NIM) odd/even.
c. Reverse the order of elements in the list.
d. Remove the element with the largest value from the list. Sort the elements in the list in descending order.

ASSIGNMENT 2

Create a tuple containing a dictionary with 10 key-value pairs of courses and grades. Among these 10 key-value pairs, ensure that at least 3 have empty values. Perform the following operations:

a. Remove incomplete entries or those with empty values.
b. Count the number of entries remaining after deletion.
c. Sort the key-value pairs by value in ascending order.
d. Find the course with the highest grade.
e. Calculate the average of all course grades.

ASSIGNMENT 3 (EVEN STUDENT ID/NIM)

Create a dataframe consisting of 10 rows and 6 columns, with the following information:

	Brand	Branch	Price (thousand)	Stock	Status
1	Avoskin	Surabaya	150	175	Sold
2	G2G	Jakarta Pusat	75	160	Available
3	Skintific	Bandung	120	120	Available
4	Wardah	Bandung	85	300	Sold
5	Emina	Jakarta	75	230	Available
6	Somethinc	Bandung	120	200	Available
7	Azarine	Surabaya	50	180	Available
8	Lacoco	Bandung	90	100	Sold
9	Y.O.U	Jakarta Pusat	55	180	Available
10	Madame Gie	Surabaya	50	200	Sold

Perform the following analysis on the DataFrame:

a. Display a summary of descriptive statistics including minimum, maximum, sum, mean, standard deviation, and quartile 1, 2, and 3 values.
b. Identify the top 3 brands with the largest revenue (price x stock).
c. Group brands with the same price.
d. Create a new column containing categories 'Cheap', 'Medium', and 'Expensive'. These categories are based on the following rules:
    - Prices below quartile 1 are categorized as 'Cheap'.
    - Prices above quartile 3 are categorized as 'Expensive'.
    - The rest are categorized as 'Medium'.

ASSIGNMENT 3 (ODD STUDENT ID/NIM)

Create a dataframe consisting of 10 rows and 6 columns, with the following information:

	Brand	Release Year	Sold	Price (million)	Types of sale
1	Apple	2015	150	19	Online
2	Xiaomi	2017	75	9	Offline
3	Samsung	2018	120	10	Online
4	Infinix	2015	85	9	Online
5	Oppo	2017	75	4	Offline
6	Asus	2020	120	9	Online
7	Lenovo	2019	50	15	Offline
8	Huawei	2019	90	15	Online
9	Google Pixel	2020	55	15	Offline
10	Sony	2017	50	11	Offline

Perform the following analysis on the DataFrame:

a. Display a summary of descriptive statistics including minimum, maximum, sum, mean, standard deviation, and quartile 1, 2, and 3 values.
b. Identify the top 3 brands with the largest revenue (price x sold).
c. Group brands by the same sales type.
d. Create a new column containing categories 'Cheap', 'Medium', and 'Expensive'. These categories are based on the following rules:
    - Prices below quartile 1 are categorized as 'Cheap'.
    - Prices above quartile 3 are categorized as 'Expensive'.
    - The rest are categorized as 'Medium'.

GRADING CRITERIA & DETAILS

Activity	Weight
Codelab 1	5%
Codelab 2	5%
Codelab 3	10%
Sub-Total	**20%**
Assignment 1	5%
Understanding of Assignment 1	20%
Assignment 2	5%
Understanding of Assignment 2	20%
Assignment 3	10%
Understanding of Assignment 3	20%
Sub-Total	**80%**
Total	**100%**